

Duofy V1 Marketing AI

Estado atual, funcionalidades, segurança, dados, qualidade e infraestrutura

Data: 2026-07-01 · **Branch:** main · **HEAD:** 9dfa586

Método: auditoria multiagente (6 frentes paralelas) com validação cruzada e evidência arquivo:linha

Escopo: backend FastAPI (~10.446 linhas) · frontend Next.js 14 (~10.195 linhas) · PostgreSQL+pgvector (24 tabelas / 15 migrations) · Docker dev+prod (Caddy/TLS)

Veredito em uma linha — Produto substancialmente real e funcional (7 telas integradas, orquestração multiagente, RAG, guardião de qualidade, stack de produção com TLS). Os débitos são saneáveis: hardening de autenticação, ~2.850 linhas de código morto e testes sem cobertura ponta a ponta. Não é caso de reescrita — é hardening, limpeza e verificação.

Conteúdo

- 00 Sumário Executivo
- 01 Arquitetura e Estado Atual
- 02 Funcionalidades e Níveis de Confiança
- 03 Segurança e Vulnerabilidades
- 04 Modelo de Dados
- 05 Qualidade, Código Morto e Testes
- 06 Infraestrutura e Deploy
- 07 Plano de Ação Priorizado

00 · Sumário Executivo

Projeto: Duofy V1 Marketing AI — plataforma interna de marketing assistido por IA (orquestrador multiagente, cocriação de conteúdo, pesquisa de mercado, calendário editorial, memória/RAG, métricas de custo e assessoria de imprensa). **Data da auditoria:** 2026-07-01 · **Branch:** `main` · **HEAD:** `9dfa586`. **Escopo:** backend (FastAPI, ~10.446 linhas Python), frontend (Next.js 14, ~10.195 linhas TS/TSX), banco (PostgreSQL + pgvector, 24 tabelas, 15 migrations), infraestrutura (Docker dev + prod com Caddy/TLS), segurança e qualidade.

1. Veredito geral

O Duofy V1 é um **MVP maduro e majoritariamente real**, não um protótipo de fachada. O núcleo funcional está implementado de verdade e conectado ponta a ponta (UI → API → banco → agentes de IA). A distância até "pronto para produção séria" é composta por três eixos claros e tratáveis:

- 1. **Segurança de autenticação** — o maior débito. JWT em cookie legível por JavaScript, sem rate limiting no login, e um acoplamento de segredos que impede rotação de chave.
- 2. **Higiene de código** — ~2.850 linhas de código morto no frontend (restos do último redesign) e helpers duplicados no backend, que inflam o repositório e confundem a leitura do estado real.
- 3. **Verificação** — a suíte de testes cobre bem lógica isolada, mas **não valida nenhum fluxo de negócio ponta a ponta**, não usa banco real, não testa auth/aprovação/worker/RAG, não tem testes de frontend e **não há CI**.

Nenhum desses eixos exige reescrita. São hardening, limpeza e cobertura — trabalho de acabamento, não de fundação.

O que está genuinamente bom (pontos fortes confirmados)

- **Orquestração de IA real:** grafo LangGraph multiagente (chat) + serviços de geração diretos, com 5 ferramentas (research, content, press, calendar, memory). Sem chave de provedor, o sistema **levanta erro amigável — não devolve dado falso**.
- **Guardião de Qualidade** sofisticado: rubrica determinística local + avaliação híbrida por LLM, com gate real antes da aprovação humana.
- **RAG com pgvector** e índices HNSW (`vector_cosine_ops`), dimensão 1536 consistente em todo o sistema.
- **Hardening de segredos fail-fast:** a API se recusa a subir em produção com segredo/senha default (`settings.py:44-65`), reforçado no `docker-compose.prod.yml` com `${VAR:?}`.
- **Stack de produção sólida na borda:** só o Caddy expõe 80/443; banco, Redis, API e worker ficam na rede interna; TLS automático via Let's Encrypt; same-origin sem CORS.
- **Frontend com tipagem forte** (zero `any` em todo `apps/web`) e 7 telas realmente integradas à API.

2. Scorecard por dimensão

Dimensão	Nota	Leitura
Arquitetura & design	8,0 / 10	Camadas serviço/router limpas; dois "cérebros" de IA bem definidos; algumas duplicações.
Funcionalidades (maturidade)	7,5 / 10	Maioria REAL; poucas PARCIAL/FRÁGIL; 1 tela MOCK (<code>/redes</code>).
Modelo de dados	8,0 / 10	Esquema coeso e consistente; dívidas de integridade (sem cascades, <code>brand_slug</code> sem FK).

Alinhamento front ↔ back	8,0 / 10	Nenhuma chamada quebrada; poucas rotas órfãs no back.
Segurança	4,5 / 10	3 críticas + 4 altas; base ok (sem SQLi, sem path traversal), mas auth é o calcanhar.
Infraestrutura & deploy	6,5 / 10	Borda sólida; bootstrap do admin não automatizado; sem backup/limites de recurso.
Qualidade de código	7,0 / 10	Limpo (zero TODO/FIXME reais), mas com código morto e duplicação.
Testes & CI	2,0 / 10	~20% de confiança; nenhum teste E2E, nenhum de frontend, nenhuma CI .
Documentação	6,0 / 10	Um doc de estado é excelente; vários outros desatualizados vs. a UI atual.
Global (ponderado)	≈ 6,3 / 10	Núcleo forte; segurança e testes puxam para baixo.

3. Top 10 riscos (priorizados)

#	Risco	Sev.	Onde	Doc
1	JWT em cookie sem HttpOnly/Secure → roubo de sessão por XSS	Crítica	<code>apps/web/lib/auth.ts</code>	015-19
2	Sem rate limiting no login + form pré-preenchido com senha default → brute-force imediato	Crítica	<code>apps/api/app/routers/auth.py</code>	21-37
3	JWT_SECRET_KEY deriva a chave Fernet → rotacionar o segredo inutiliza todas as API keys cifradas	Crítica	<code>apps/api/app/crypto.py</code>	11-14
4	Seed do admin não roda no boot → deploy limpo fica sem login (contradiz o DEPLOY.md)	Alta (op.)	<code>apps/api/app/seed.py</code> <code>/main.py:41-54</code>	0152
5	IDOR/BOLA em documentos → qualquer usuário baixa/exporta documento de qualquer marca	Alta	<code>apps/api/app/routers/documents.py</code>	51-59, 221-253
6	Next.js 14.2.35 com CVEs (inclui bypass de middleware) enquanto a proteção de rotas depende do middleware	Alta	<code>apps/web/package.json</code>	0312
7	Nenhum teste E2E/integração/CI → regressão silenciosa no caminho crítico	Alta	<code>apps/api/tests/*</code> , ausência de <code>.github/workflows</code>	05

8	SSRF no research (fetch de URLs arbitrárias + Chromium <code>--no-sandbox</code> , container root)	Média	<code>apps/api/app/research</code> <code>h3service.py:109-137</code>
9	Sem backup automatizado do Postgres + sem limites de recurso (risco de OOM na VPS)	Média	<code>DEPLOY.md §4</code> , <code>06</code> <code>docker-compose.prod.yml</code>
10	Fallback de embeddings local (hash SHA-256) degrada o RAG silenciosamente sem chave	Média	<code>apps/api/app/embeddings.py:29-41</code>

4. O código está alinhado? Tem código desnecessário?

Alinhado: sim, no contrato front↔back — **nenhuma chamada do frontend aponta para rota inexistente**, e a tipagem em `lib/api.ts` casa com os `response_model` do backend. Nota de alinhamento: **8,0/10**.

Código desnecessário: sim, e é significativo — mas concentrado e de baixo risco para remover:

- **Frontend (~2.850 linhas mortas):** `lib/mock.ts` (489 linhas, zero importadores) + cluster de 5 componentes órfãos do redesign anterior (`chat-panel`, `kanban-board`, `card-popup`, `inspector-bar`, `document-workspace` — este último só exporta um tipo ainda usado). Mais 7 páginas que viraram `redirect` e 2 páginas admin funcionais porém **sem link na navegação** (`/admin/agents`, `/admin/config`).
- **Backend:** `pdf_service.py` (wrapper órfão), funções redefinidas/sombreadas em `routers/outputs.py`, e `_provider_for_model` duplicado em 5 arquivos (com uma divergência de comportamento).
- **Rotas de API sem consumidor:** router `press` inteiro, `PATCH /api/agents/runs/{id}/status`, endpoints `/pdf` dedicados e os de manutenção `reformat-legacy/repair-formatting`.
- **Bug de estilo:** a classe Tailwind `surface` é usada 17× em 5 arquivos mas **não existe** — silenciosamente ignorada.

Detalhe importante: `lib/mock.ts` **não está mais ligado a nenhuma tela**. A única tela mock (`/redes`) define seus próprios dados hardcoded localmente. Ou seja, "o sistema usa mock" é hoje **falso**, exceto por `/redes`.

5. Recomendação de sequência (resumo)

1. **Sprint de segurança (bloqueadores):** cookie `HttpOnly/Secure`, rate limiting no login, separar `JWT_SECRET_KEY` da chave de criptografia, escopar documentos por marca, atualizar Next.js. → **03**
2. **Sprint de bootstrap/produção:** automatizar o seed no `migrate`, backup em cron, limites de memória, healthcheck do worker, non-root no container da API. → **06**
3. **Sprint de higiene:** deletar as ~2.850 linhas mortas, consolidar helpers duplicados, corrigir a classe `surface`, decidir sobre páginas órfãs. → **05**
4. **Sprint de verificação:** CI mínima (`ruff` + `pytest` + `next lint/build`) + `conftest.py` com banco de teste + testes E2E de login e do fluxo de aprovação. → **05**

O plano detalhado, com esforço estimado por item, está em **07 · Plano de Ação Priorizado**.

01 · Arquitetura e Estado Atual

1. Visão de sistema

O Duofy V1 é uma aplicação **full-stack local-first / single-tenant** (uma equipe interna, múltiplas marcas), organizada como monorepo:

```
DUOFY_V1_MARKETING_AI/
├── apps/
│   ├── api/                FastAPI (Python 3.11) — ~10.446 linhas
│   │   └── app/
│   │       ├── routers/    16 routers HTTP
│   │       ├── orchestrator* grafo LangGraph + single-agent + tools + llm
│   │       ├── *_service.py serviços de domínio (content, research, calendar, metrics, operations...)
│   │       ├── quality_guardian.py, rag.py, embeddings.py, llm.py, crypto.py ...
│   │       ├── models.py, schemas.py, db.py, settings.py, seed.py
│   │       └── worker.py    Celery
│   └── alembic/versions/    15 migrations (0001-0015)
├── web/                    Next.js 14 App Router (TypeScript) — ~10.195 linhas
│   ├── app/(app)/*/*page.tsx 17 rotas (7 reais, 1 mock, 9 redirect/órfãos)
│   ├── components/           UI + cluster órfão
│   ├── lib/                  api.ts, auth.ts, brand-context.tsx, mock.ts...
│   └── middleware.ts          proteção de rotas
├── config/                  prompts de agentes, marcas, templates, regras (Markdown/YAML)
├── infra/                   postgres init (pgvector), Caddy
├── docker-compose.yml (dev) · docker-compose.prod.yml (prod)
└── docs/                    documentação (parcialmente desatualizada)
```

Stack tecnológica

Camada	Tecnologia
Frontend	Next.js 14.2.35 (App Router), React 18.3, TypeScript 5, Tailwind 3.4
Backend	FastAPI, SQLAlchemy 2 (async / asyncpg), Pydantic v2, Uvicorn
IA / Orquestração	LangGraph + LangChain ChatOpenAI (grafo); httpx direto (serviços)
Provedores LLM	OpenRouter, OpenAI, Anthropic; OpenAI embeddings; Apify (scraping)
Banco	PostgreSQL 16 + extensão pgvector (embeddings 1536-D, índices HNSW)
Fila / Cache	Redis 7 (broker + backend Celery; lock do scheduler)
Assíncrono	Celery (--pool=solo)
Export/PDF	WeasyPrint (Pango/Cairo) + gerador Markdown→HTML próprio
Scraping	Google News RSS + trafilatura/BeautifulSoup + Playwright (Chromium)
Reverse proxy / TLS	Caddy 2 (Let's Encrypt automático) — só em produção
Empacotamento	Docker Compose (stacks separadas dev e prod)

2. Topologia de serviços

Desenvolvimento (docker-compose.yml)

```
[host:5433]→postgres(pgvector)    [host:6379]→redis
      ■                ■
migrate (alembic upgrade head, one-shot)
      ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
api(:8000) worker(celery solo)
      ■
web(:3000, next dev, hot-reload)
```

- postgres e redis fazem bind apenas em `127.0.0.1` (não expostos na LAN).
- Ordem de boot determinística: `postgres/redis healthy` → `migrate completed` → `api/worker` → `web` (espera `api healthy`).
- `web` monta o código-fonte do host por volume (hot-reload): `node_modules/.next` em volumes nomeados.

Produção (docker-compose.prod.yml)

```

Internet
  ■ 443/80
  ■■■■■■■■■■
  ■ Caddy ■ (TLS automático, same-origin)
  ■■■■■■■■■■
/api/* /health ■ resto
  ■■■■■■■■■■ ■■■■■■■■■■
api(:8000) web(:3000, next standalone non-root)
  ■ ■
postgres redis worker(celery solo)
(rede interna – nenhuma porta publicada)

```

- **Só o Caddy publica portas.** Postgres, Redis, API, worker e web ficam exclusivamente na rede interna do Docker.
- Frontend chama `/api` relativo → roteado pelo Caddy para `api:8000` → **same-origin, sem CORS.**
- Segredos são **obrigatórios** (`${VAR:?}`) e validados pela API no boot.

Detalhes completos, healthchecks e checklist de produção em **06 · Infraestrutura e Deploy**.

3. Os dois "cérebros" de IA (ponto arquitetural central)

O sistema tem **dois caminhos de execução de IA distintos e coexistentes** — não é duplicação, são propósitos diferentes:

(A) Orquestrador multiagente — LangGraph (assíncrono, via chat)

- **Arquivos:** `orchestrator_graph.py` (grafo `run_orchestrator`), `orchestrator_llm.py` (fábrica `ChatOpenAI` + callback de métricas), `orchestrator_tools.py` (5 ferramentas).
- **Fluxo:** `routers/chat.py` cria um `AgentTask` e dispara Celery (`.delay`) → `worker.py` → `task_service.py` → `run_orchestrator`. Loop `agent` → `tools` → `final` com `MAX_STEPS=5`. As tools chamam os serviços de domínio (research/content/press/calendar/memory), cada uma protegida por `_with_rollback` para não envenenar a sessão async compartilhada.
- **Este é o único fluxo verdadeiramente offloaded para o worker.**
- **Fragilidades:** o checkpointer é `MemorySaver` (estado em memória do worker, perdido no restart — migração para `AsyncPostgresSaver` está documentada mas não implementada); o `ChatOpenAI` assume base OpenAI-compatible (OpenRouter por default), então provider `anthropic` nativo não funcionaria como esperado — mascarado pelo default do seed (`~anthropic/claude-sonnet-latest` via OpenRouter).

(B) Serviços de geração diretos — httpx (síncrono, via REST)

- **Arquivos:** `content_generation.py`, `research_service.py`, `calendar_service.py`, `quality_guardian.py`, chamando `llm.py::call_llm`.
- **Fluxo:** os routers REST (`/api/content/generate`, `/api/research/run`, `/api/calendar/generate`, `/api/press/generate`, revisão de qualidade) chamam esses serviços **sincronamente dentro do request HTTP**. As mesmas funções também são as ferramentas do grafo (A).
- **Também roda em `orchestrator.py::run_agent`** (single-agent, sem tools), usado por `POST /api/agents/run` e como fallback do calendário.

Implicação de desempenho

Apenas o chat é assíncrono. Toda geração de conteúdo, pesquisa, imprensa, calendário, revisão de qualidade, upload/indexação de documento e geração de relatório **bloqueia o request HTTP** (com `run_in_threadpool` apenas para trechos CPU-bound como export e extração de texto). Uma geração pode segurar a conexão por dezenas de segundos.

O scheduler de calendário (terceiro caminho, frágil)

`calendar_scheduler.py` roda um loop `asyncio` iniciado no **lifespan do processo web/API** (`main.py:43`) — **não é Celery**. Poll de 60s, lock Redis `SETNX` por evento, executa até 5 eventos vencidos por tick, chamando `execute_calendar_event` **sincronamente** no processo da API. Deveria idealmente ser uma Celery beat task. Ver 02 · Funcionalidades.

4. Fluxos de dados principais

Login

```
login-form.tsx → POST /api/auth/login → security.py (PBKDF2 verify)
→ JWT HS256 (12h) → cookie duofy_token (client-side, NÃO HttpOnly)
→ middleware.ts protege rotas checando presença do cookie
```

Geração de conteúdo (síncrona)

```
/operations → POST /api/content/generate → content_generation.generate_content_output
→ seleciona template (canal/formato) → build_rag_context (pgvector) → call_llm (httpx)
→ normaliza documento → grava AgentRun + Output + OutputVersion(v1) → registra ModelCall (custo)
```

Chat com orquestrador (assíncrona)

```
/operations (chat) → POST /api/chat/.../messages → cria AgentTask(queued) → Celery .delay
→ worker → run_orchestrator (LangGraph) → tools (research/content/...) → resposta
→ /api/tasks/{id}/stream (SSE por polling 1s, teto 3 min)
```

RAG / memória

```
upload → /api/documents/upload → extrai texto (pypdf/python-docx) → chunk (900/140)
→ embed_text por chunk (OpenAI ou fallback local) → grava document_chunks(embedding vector 1536)
busca → /api/memory/search → embed da query → SQL UNION ALL (chunks + memory_entries)
→ similaridade cosseno pgvector (<=>) → filtros marca/categoria → top-k
```

Aprovação de output (workflow)


```
gerar → submit-review → Guardião de Qualidade (gate: score ≥ 80, sem falhas críticas)
→ /approvals → approve (exige revisão aprovada; cria memória permanente)
| reject (cria memória temporária, expira 30 dias)
| request-adjustment | archive | move (kanban)
```

5. Configuração externa (Markdown/YAML)

Um diferencial de design: prompts de agentes, perfis de marca, templates de conteúdo e regras vivem em `config/` como arquivos versionados, carregados em runtime:

- `config/agents/*.md` — prompts dos 7 agentes (orquestrador, pesquisa, conteúdo, calendário, imprensa, métricas, guardião).
- `config/brands/*.md` + `config/seeds/brands.yaml` — voz de marca por marca, injetada nas skills.
- `config/templates/*.md` — templates por canal (instagram, linkedin, carousel, press release, research report).
- `config/rules/*.yaml` — `output_status.yaml` (máquina de estados), taxonomia RAG, matriz de personas/objeções, pricing de modelos.

Há 3 implementações independentes de descoberta desse diretório (`agent_config.read_config_text`, `metrics._pricing_config`, `output_workflow._load_status_rules`), todas com o mesmo fallback para `DUOFY_V1_pacote_execucao_desenvolvimento/` — candidato a consolidação (ver 05).

6. Os 7 agentes

Agente	Slug	Papel	Estado
Orquestrador	<code>orchestrator</code>	Roteia intenção do chat para ferramentas	REAL (grafo)
Pesquisa & Inteligência	<code>research_agent</code>	Coleta e classifica fontes de mercado	PARCIAL/FRÁGIL (depende de RSS/Playwright externos)
Cocriação & Conteúdo	<code>content_agent</code>	Gera conteúdo versionado por canal	REAL
Calendário & Campanhas	<code>calendar_agent</code>	Planeja e executa eventos editoriais	REAL
Assessoria de Imprensa	<code>press_agent</code>	Gera releases/materiais de imprensa	REAL no back, sem UI
Métricas & Análise Interna	<code>metrics_agent</code>	Relatórios internos de custo/uso	REAL
Guardião de Qualidade	<code>quality_guardian</code>	Gate de qualidade antes da aprovação	REAL (o módulo mais sofisticado)

7. Estado por camada (resumo)

Camada	Maturidade	Observação
Backend / API	REAL	16 routers, orquestração real, sem dados falsos (erro amigável sem chave).
Modelo de dados	REAL	24 tabelas coesas; dívidas de integridade (sem cascades, <code>brand_slug</code> sem FK).
Frontend	REAL (7 telas)	1 tela mock (<code>/redes</code>); ~2.850 linhas de código morto.

Orquestração de IA	REAL / PARCIAL	Grafo funcional; estado volátil (MemorySaver); só chat é assíncrono.
RAG / embeddings	PARCIAL	Infra vetorial correta; query pode não usar o índice HNSW; fallback local degrada.
Infraestrutura	PARCIAL	Borda sólida; bootstrap do admin não automatizado; sem backup/limites.
Segurança	FRÁGIL	Auth é o calcanhar (ver 03).
Testes / CI	FRÁGIL	~20% de confiança; nenhum E2E; nenhuma CI.

Continue por **02 · Funcionalidades e Níveis de Confiança** para o inventário detalhado feature a feature.

02 · Funcionalidades e Níveis de Confiança

Inventário completo das funcionalidades, classificadas por maturidade: **REAL** (completo e funcional) · **PARCIAL** (funciona com lacunas) · **FRÁGIL** (alto risco de quebrar) · **MOCK/STUB** (placeholder/dados fictícios).

1. Backend — funcionalidades

1.1 Autenticação — REAL

- **Endpoints:** `POST /api/auth/login`, `POST /api/auth/logout`, `GET /api/auth/me`.
- **Como funciona:** senha verificada com PBKDF2-HMAC-SHA256 (390k iterações, salt por senha, comparação em tempo constante — `security.py:15-34`). JWT HS256 assinado com `jwt_secret_key`, expiração 12h. `get_current_user` revalida `is_active` a cada request (`dependencies.py:34-36`).
- **Ressalvas:** logout é no-op (JWT stateless, sem revogação); token vive 12h sem denylist. Segurança do transporte do token é frágil (ver 03, C-1/A-1).

1.2 Marcas — REAL (read-only)

- `GET /api/brands` lista marcas ativas do seed. **Não há CRUD de marca via API** — marcas vêm do seed/YAML.

1.3 Admin / Configuração de provedores — REAL

- **Endpoints:** `GET /api/admin/agents`, `GET/PUT /api/admin/providers/{provider}`, `GET/PUT /api/admin/quality-settings`, `GET/PUT /api/admin/agent-settings`.
- **Como funciona:** a chave de API do provedor é **cifrada** com Fernet ao salvar (`admin.py:216-217`) e **mascarada** na leitura (`abcd...wxyz`) — o segredo nunca é devolvido em claro. Configura orçamentos de token, profundidade de pesquisa e modo do Guardião (`local/hybrid/llm_required`).

1.4 Geração de conteúdo — REAL

- **Endpoints:** `POST /api/content/generate`, `GET /api/content/outputs`, `GET /outputs/{id}`, `PATCH /outputs/{id}`, `POST /outputs/{id}/submit-review`.
- **Como funciona:** seleciona template por canal/formato → monta contexto RAG → chama LLM → normaliza documento → grava `AgentRun` + `Output` + `OutputVersion` v1. Tratamento de erro robusto com rollback e registro de `AgentRun` falho (`content_generation.py:214-227`).

1.5 Workflow de outputs / versões / comentários — REAL (com código morto no router)

- **Como funciona:** máquina de estados dirigida por `config/rules/output_status.yaml` (com fallback embutido). `approve` exige `ensure_quality_passed` (gate do Guardião) e cria memória permanente; `reject` cria memória temporária (30 dias); há `request_adjustment`, `archive`, `move` (kanban), versões, diff (`diffLib`), restore e comentários ancorados.
- **Dívida:** `routers/outputs.py` tem funções **redefinidas/sombreadas** (`_version_read`, `_output_read`, `_output_export_document` definidas duas vezes — a primeira definição é código morto). Ver 05.

1.6 Guardião de Qualidade — REAL (módulo mais sofisticado)

- **Como funciona:** avaliação em dois níveis. **Local determinístico** (`assess_output_quality`): rubrica com penalidades por mojibake, placeholder, tamanho, ausência de seções/CTA/fontes, e regras de segurança de marca hardcoded (ex.: não misturar DeathCare↔Postos, tom sensível). `Score = 100 - penalidades`; aprova `≥80` sem críticas. **Híbrido opcional por LLM** (3 modos): merge conservador (`score = min(local, llm)`, `passed` exige

aprovação do LLM). Erro de LLM é capturado sem derrubar a revisão.

1.7 Pesquisa de mercado — PARCIAL / FRÁGIL

- **Endpoint:** `POST /api/research/run`.
- **Como funciona:** pipeline real de coleta — Google News RSS + fontes manuais + Apify opcional; extração via `trafilatura/BeautifulSoup`; fallback **Playwright/Chromium** para profundidade `deep`. Classifica confiabilidade A–D por domínio. Fontes que falham viram registros `status="failed"` (degradação graciosa).
- **Por que FRÁGIL:** depende de RSS externo do Google News (quebra fácil, sem contrato de API); Playwright exige o browser instalado no container; `_apify_candidates` engole exceções silenciosamente retornando `[]` (`research_service.py:204-205`). Ver também SSRF/no-sandbox em 03.

1.8 Assessoria de imprensa — REAL no back, sem UI

- **Endpoint:** `POST /api/press/generate` (análogo a conteúdo; pode vincular a um `CalendarEvent`). **Nenhuma tela do frontend consome** este router — funcionalidade existe só no backend.

1.9 Calendário editorial — REAL

- **Endpoints:** `GET/POST /api/calendar`, `PATCH/DELETE /{id}`, `POST /generate`, `POST /{id}/run-now`, `GET /export.ics`.
- **Como funciona:** CRUD completo; `generate` pede ao LLM um JSON de 3–8 eventos com parse defensivo (`_extract_json_array`, `_parse_datetime` degrada data malformada). `execute_calendar_event` roteia por `assigned_agent_slug` para press/content/research (fallback genérico via `run_agent`), tratando falha com re-fetch pós-rollback. Geração de **iCal RFC 5545** sem dependências (line-folding + escaping corretos).

1.10 Scheduler de calendário — FRÁGIL

- **Como funciona:** loop `asyncio` no lifespan (`main.py:43`), poll 60s, lock Redis `SETNX` por evento, executa até 5 eventos vencidos por tick.
- **Por que FRÁGIL:** roda **no processo da API, não no worker**; múltiplas réplicas web rodariam o loop (mitigado pelo lock, mas ainda executa geração LLM pesada no processo web); execução síncrona dentro do tick. Deveria ser Celery beat.

1.11 Chat / Tarefas assíncronas — REAL

- **Endpoints:** `POST /api/chat/sessions/{id}/messages`, `GET /api/tasks, /{id}, /{id}/stream` (SSE por polling 1s, teto 180 iterações = 3 min).
- **Como funciona:** cria `AgentTask(queued)` e dispara Celery → orquestrador LangGraph. Sessões de chat são **escopadas por usuário** (`ChatSession.user_id == user.id`). Cada iteração do SSE abre nova sessão DB (custo de conexões).
- **Nota:** namespaces `task_type="orchestrate"` (chat) vs `"orchestrator"` (métricas) divergem, mas nada filtra por eles em V1.

1.12 RAG / Memória / Embeddings — PARCIAL

- **Endpoints:** `/api/memory`, `POST /api/memory/search`.
- **Como funciona:** `search_memory` faz SQL `UNION ALL` sobre `document_chunks` + `memory_entries` com similaridade cosseno pgvector (`<=>`), filtros por marca/categoria/fonte, respeitando `expires_at`.
- **Ponto crítico 1 (STUB silencioso):** sem provedor `openai_embeddings/openai` habilitado, `embed_text` cai para `_local_embedding` — **hashing SHA-256 por token** em vetor 1536-D (`embeddings.py:29-41`). Não é embedding semântico: o RAG funciona mecanicamente mas com qualidade de recuperação ruim, **sem avisar o usuário**. É o

principal componente "mock-like" do sistema.

- **Ponto crítico 2 (performance):** a query de busca ordena por score sobre um `UNION ALL` externo (`rag.py:55-91`), padrão que **provavelmente impede o Postgres de usar o índice HNSW** criado na migration 0015 — a confirmar com `EXPLAIN ANALYZE`. Ver 04.
- **Ponto crítico 3:** `embed_text` **não tem try/except nem retry** (`embeddings.py:68-77`) — se o provedor de embeddings estiver habilitado mas falhar/timeout, o erro propaga cru (500), diferente do resto que degrada.

1.13 Documentos — REAL

- **Endpoints:** `POST /upload` (PDF/DOCX/TXT/MD), `GET` lista/chunks/download/export.
- **Como funciona:** extrai texto (pypdf/python-docx), chunking (900/140), embedding por chunk, indexação. Processamento **síncrono dentro do request** via `run_in_threadpool` — lento para arquivos grandes (sem fila).
- **Falha de UI:** não há tela de **upload** no frontend (a `/memory` só lê/baixa) — lacuna funcional.
- **Falha de segurança:** rotas não escopam por marca/usuário (IDOR — ver 03, A-2).

1.14 Métricas e Relatórios — REAL

- **Endpoints:** `GET /api/metrics/summary`, `/model-calls`; `POST /api/reports/generate`.
- **Como funciona:** `record_model_call` grava **toda** chamada LLM (sucesso e falha) numa sessão DB própria — desacoplada da transação do request, para não perder telemetria em rollback. Custo estimado por `config/rules/model_pricing.yaml` ou `raw_usage.cost` do OpenRouter. Relatórios internos com export PDF/DOCX/MD/HTML.

1.15 Operações / Auditoria — REAL

- **Endpoints:** summary agregado, `agent-health` (classifica ok/warning/critical por taxa de falha), listagem de audit events e quality reviews.
- **Como funciona:** `record_audit_event` compacta metadata grande; agrega model calls, agent runs, quality reviews e decisões.

1.16 Export / PDF — REAL (depende de libs nativas)

- **Como funciona:** gerador Markdown→HTML próprio (listas, tabelas, blockquote, inline) em 4 formatos; **PDF via WeasyPrint** (import tardio). Exige Pango/Cairo no container — **presentes** no `apps/api/Dockerfile:9-19` (confirmado). `pdf_service.py` é um wrapper fino redundante sobre `export_service` (candidato a remoção).

1.17 Integração com provedores de IA — REAL / FRÁGIL

- **Suportados:** `openrouter`, `openai`, `anthropic` (execução), `openai_embeddings` (RAG), `apify` (scraping).
- **Chaves:** cifradas em `provider_credentials.api_key_encrypted`, decifradas via Fernet derivado do `JWT_SECRET_KEY` (acoplamento crítico — ver 03, C-3).
- **Sem chave:** levanta `LLMConfigurationError` → HTTP 400/422 com mensagem "Configure e habilite o provedor... em Admin > Configurações > Modelos LLM". **Nenhuma geração devolve dado falso.**
- **FRÁGIL — override de modelo:** todos os serviços chamam `call_llm(model=credential.default_model or model, ...)`. Ou seja, o `default_model` do provedor **sobrepõe** o modelo escolhido pelo usuário/agente sempre que estiver setado (o padrão pós-seed). O parâmetro `model` do usuário é efetivamente ignorado — comportamento de negócio não documentado.

2. Frontend — telas

Legenda de nav: os rótulos entre parênteses são os exibidos na navegação.

Tela / Rota	Rótulo	Classificação	Confiança	Observações
/login	—	REAL	Alta	Remover credenciais default pré-preenchidas no form.
/operations	Operações	REAL	Alta	Tela central monolítica (527 linhas): chat + kanban drag-drop + cocriação + modal de edição.
/approvals	Revisão	REAL	Alta	Fila de revisão + painel do Guardião. UX de erro crua (alert/prompt).
/calendar	Calendário	REAL	Muito alta	CRUD + geração IA + .ics + deep-links Google/Outlook.
/memory	Memória	REAL	Alta	Busca RAG, coleções, chunks, download. Falta upload .
/relatorios	Relatórios	REAL	Muito alta	Custos/uso de IA com gráficos derivados de dados reais.
/admin	Administração	REAL*	Alta	6 abas; *Permissões é mock, Integrações é parcial (só Apify real).
/admin/agents	—	REAL, órfã	Média	Funcional, mas sem link na nav ; duplica /admin . Candidata a remoção.
/admin/config	—	REAL, órfã	Média	Idem acima.
/redes	Redes & Tráfego	MOCK	Baixa	100% hardcoded (ORGANIC_KPIS, ADS...). Sem apiFetch/useBrand . Botões inertes.
/dashboard /chat /content /research /workspace	—	Redirect	N/A	Só redirect("/operations") .
/costs /insights	—	Redirect	N/A	Só redirect("/relatorios") .

Detalhamento das telas mock/parciais

- **/redes (MOCK)**: dashboard de Instagram Insights + Meta Ads com dados fictícios definidos localmente. É a **única** tela puramente mock. Coerente com o escopo V1 (publicação em redes fora de escopo). Precisa de backend real (Meta/Instagram APIs) para sair de mock.
- **/admin > aba Permissões (MOCK)**: **ROLE_TABLE** hardcoded; botão "Novo papel" desabilitado ("Em breve").
- **/admin > aba Integrações (PARCIAL)**: Apify é real (deriva de providers); Meta Ads / Google Sheets / Tavily / SendGrid são placeholders "Em breve".
- **/admin > aba Automações (PARCIAL)**: monitor de execuções é real; os "Fluxos" são rótulos hardcoded decorativos.

Transversais do frontend

- **Autenticação:** token em cookie `duofy_token` não-HttpOnly, sem Secure (`lib/auth.ts:15-19`), enviado como Bearer manual em cada chamada. Sem tratamento **centralizado de 401** — telas com token expirado mostram "vazio" em vez de deslogar.
- **Multi-marca:** `brand-context.tsx` provê `{brands, selected, setSelected}`, persistido em `localStorage`. Propagação **mista**: algumas telas passam `?brand_slug=` (server-side), outras filtram no cliente (frágil se houver mais itens que o `limit`).
- **Resiliência de fetch inconsistente:** `Promise.allSettled` (resiliente) em memory/relatorios/admin vs `Promise.all` (falha tudo) em operations/admin-config. A versão resiliente deveria ser padrão.
- **Bug de estilo:** classe Tailwind `surface` usada 17× em 5 arquivos, **não existe** — silenciosamente ignorada (hovers/backgrounds não aparecem).

3. Resumo — o que é real vs. o que não é

É real e funcional (ponta a ponta): login, operações (chat + kanban + cocriação), revisão com Guardião, calendário editorial, memória/RAG (busca), relatórios de custo, administração de provedores/agentes/qualidade, geração de conteúdo, métricas, auditoria, export PDF/DOCX/MD/HTML.

Funciona com ressalva (PARCIAL/FRÁGIL): pesquisa de mercado (scraping externo), scheduler de calendário (in-process), RAG (índice possivelmente não usado + fallback de embedding degradado), override de modelo (ignora escolha do usuário), orquestrador (estado volátil).

Não é real (MOCK/ausente): tela `/redes` (100% mock), aba Permissões do admin (mock), integrações Meta/Sheets/Tavily/SendGrid (placeholders), assessoria de imprensa (existe no back, **sem UI**), upload de documentos (existe no back, **sem UI**).

Ponto que merece destaque: sem chave de provedor de LLM, **o sistema não finge** — ele levanta erro amigável em toda geração. O único comportamento "stub" silencioso é o fallback de embeddings local, que degrada o RAG sem avisar.

Continue por **03 · Segurança e Vulnerabilidades**.

03 · Segurança e Vulnerabilidades

Auditoria AppSec do projeto (FastAPI + Next.js + Postgres + Redis/Celery + Docker). Todo achado tem evidência `arquivo:linha`.

Contexto atenuante: a aplicação é **single-tenant, para uma equipe interna única**, e em produção Postgres/Redis/API/worker ficam só na rede interna do Docker (apenas o Caddy expõe 80/443). Isso reduz — mas não elimina — o impacto de vários achados. Um comprometimento de qualquer estação da equipe (via XSS) ainda é caminho direto para roubo de sessão.

Mapa de severidade

Sev	Achados
CRÍTICA	C-1 JWT cookie sem HttpOnly/Secure · C-2 sem rate limit no login · C-3 acoplamento JWT↔Fernet
ALTA	A-1 sem revogação + expiração 12h · A-2 IDOR em documentos · A-3 admin default fraco · A-4 Next.js com CVEs
MÉDIA	M-1 SSRF research · M-2 SSRF/roubo de key via base_url · M-3 middleware só checa presença · M-4 container API root · M-5 Chromium - -no-sandbox · M-6 CORS+credentials
BAIXA	B-1 Content-Disposition · B-2 log de dados sensíveis · B-3 PBKDF2 (ok, mas não argon2) · B-4 sem HSTS/CSP

Ordem de correção recomendada: C-1 e C-2 (exploráveis remotamente sem privilégio) → A-4 (bypass de middleware Next) → C-3 (separar segredos antes de qualquer rotação) → A-2 (autorização de objeto).

CRÍTICA

C-1 · JWT em cookie sem HttpOnly / Secure (roubo de sessão via XSS)

Categoria: OWASP A07 Broken Auth / A05 Misconfiguration **Evidência:** `apps/web/lib/auth.ts:15-19` — `document.cookie = "duofy_token=...; path=/; max-age=43200; SameSite=Lax"`. Sem `HttpOnly`, sem `Secure`. O token é lido por JS em `getTokenFromCookie` (`lib/auth.ts:3-13`) e salvo no cliente após login (`login-form.tsx:27`). **Impacto:** qualquer XSS (ou dependência JS comprometida) lê o JWT em claro e o exfiltra. Com validade de 12h e sem revogação (A-1), o atacante assume a sessão — inclusive admin. A ausência de `Secure` permite vazamento em qualquer request HTTP (o `.env.production.example` chega a prever `:80/HTTP` puro para testes). **Correção:** emitir o token como cookie `HttpOnly; Secure; SameSite=Strict` no `Set-Cookie` do `/login` (backend) e lê-lo server-side. O `middleware.ts` já lê o cookie no servidor, então a migração é viável; o obstáculo é o `getTokenFromCookie()` espalhado por ~15 telas. **Nota:** já registrado na memória do projeto como dívida consciente (`jwt-cookie-httponly-deferred`) — mas **continua sendo o risco de maior severidade**.

C-2 · Ausência total de rate limiting / anti-brute-force no login

Categoria: OWASP A07 Identification & Auth Failures **Evidência:** `apps/api/app/routers/auth.py:21-37` — `/api/auth/login` sem throttling/lockout/captcha. Busca por `slowapi|Limiter|rate_limit` no backend só encontra `agent_limits.py` (orçamento de tokens de LLM) — **não há rate limiting HTTP em nenhuma rota**. **Impacto:** brute-force/credential-stuffing sem fricção. Agravado por: senha admin default (A-3) e login form **pré-preenchido** com `admin@duofy.com.br / admin123456` (`login-form.tsx:12-13`). Também abre DoS por tentativa massiva. **Correção:** rate limiting por IP+conta no `/login` (ex.: `slowapi`, ou no próprio Caddy), lockout progressivo, log de tentativas falhas.

C-3 · JWT_SECRET_KEY deriva a chave Fernet (segredo único, dois papéis)

Categoria: OWASP A02 Cryptographic Failures / Secrets Management **Evidência:** `apps/api/app/crypto.py:11-14` — a chave Fernet é derivada por SHA-256 do próprio `jwt_secret_key`:

```
digest = hashlib.sha256(get_settings().jwt_secret_key.encode()).digest()
key = base64.urlsafe_b64encode(digest)
return Fernet(key)
```

O mesmo segredo assina/verifica JWTs (`security.py:39-55`) e cifra as API keys dos provedores (`admin.py:217`, `llm.py:51`, `embeddings.py:59`, `research_service.py:183`). **Impacto duplo:** 1. **Rotação impossível sem quebra:** rotacionar `JWT_SECRET_KEY` (resposta padrão a vazamento de token) torna todas as `api_key_encrypted` indecifráveis e derruba todos os agentes. O mecanismo de resposta a incidente de auth **quebra o mecanismo de cripto de segredos**. 2. **Blast radius ampliado:** vazou o segredo do JWT = forjar tokens **E** decifrar todas as chaves de API. **Correção:** separar em dois segredos independentes (`JWT_SECRET_KEY` e `SECRETS_ENCRYPTION_KEY` via `Fernet.generate_key()`) e suportar rotação com re-cifragem das credenciais. **Fazer isso antes de qualquer rotação**, senão a rotação vira incidente. **Nota:** coerente com a memória do projeto (`jwt-secret-fernet-coupling`).

ALTA

A-1 · Sem revogação / logout real; expiração longa (12h)

Categoria: OWASP A07 Broken Auth **Evidência:** `/logout` só retorna `{"status":"ok"}` (`auth.py:40-42`); `access_token_expire_minutes = 720` (`settings.py:28`); logout do front só apaga o cookie (`lib/auth.ts:21-23`). **Impacto:** token roubado (via C-1) permanece válido até 12h mesmo após "logout" ou desativação do usuário. Mitigado parcialmente por `get_current_user` revalidar `is_active`, mas não há denylist nem versão de credencial. **Correção:** reduzir expiração (30–60 min) + refresh token; `jti/token_version` com denylist em Redis para revogação imediata.

A-2 · IDOR/BOLA — qualquer usuário baixa/exporta documento de qualquer marca

Categoria: OWASP A01 Broken Access Control **Evidência:** `apps/api/app/routers/documents.py:51-59` (`_get_document_or_404` busca só por `Document.id`) e 221-238/241-253 (download/export exigem só `get_current_user`, sem checar propriedade/marca). Contraste: o chat **escopa** por `user_id` (`chat.py:93,111`) — o padrão existe, mas não foi aplicado a documentos. **Impacto:** um `manager` enumera IDs sequenciais e baixa todo o material de memória/marca de qualquer cliente. Em contexto multi-marca, é vazamento cruzado. **Correção:** filtrar por marca/permissão em todas as rotas de documento (download, export, chunks, list) — ou documentar explicitamente o modelo "toda a equipe vê tudo".

A-3 · Credenciais admin default fracas e propagadas

Categoria: OWASP A07 / Secrets Management **Evidência:** `settings.py:10,31` (`DEFAULT_ADMIN_PASSWORD="admin123456"`), `docker-compose.yml:68-69,135-137`, `seed.py:29-48`, e o login form pré-preenchido (`login-form.tsx:12-13`). **Mitigação existente (boa):** `settings.py:44-65` faz a API **recusar subir** em `APP_ENV` não-dev se `ADMIN_PASSWORD` ainda for o default. Depende de `APP_ENV` estar correto. **Correção:** manter o hardening; remover o pré-preenchimento de senha no login; forçar troca no primeiro login; nunca expor a porta 8000 usando o compose de dev.

A-4 · Next.js 14.2.35 com CVEs conhecidos (inclui bypass de middleware)

Categoria: OWASP A06 Vulnerable & Outdated Components **Evidência:** `apps/web/package.json:12,20` — `next` e `eslint-config-next` em 14.2.35. **Impacto:** a linha 14.2.x acumulou vulnerabilidades relevantes (SSRF em Server Actions, cache poisoning e a família **CVE-2025-29927** — **bypass de autorização no middleware**). Como toda a proteção de rotas do frontend depende de `middleware.ts` (ver M-3), bypass de middleware é diretamente relevante aqui. **Correção:** rodar `npm audit`, atualizar para a última 14.2.x/15.x patcheada **em tarefa isolada com regressão**; garantir defesa em profundidade (o backend já valida o token, o que ajuda). **Nota:** já registrado na memória (`npm-audit-next-bloqueador-producao`).

MÉDIA

M-1 · SSRF no serviço de research

Evidência: `research_service.py:279-287` (URLs do usuário viram candidatos), `:109-117` (`_fetch_url_text` com `httpx.AsyncClient(follow_redirects=True)` sem validar host/esquema/IP), `:120-137` (`_fetch_with_playwright` idem). **Impacto:** a API pode ser induzida a requisitar `http://169.254.169.254/` (metadata cloud), serviços internos (`http://postgres:5432, redis`) ou `file:///`. `follow_redirects=True` permite bypass por redirect. Em VPS sem metadata endpoint o risco cai, mas a rede interna do Docker é alcançável. **Correção:** validar esquema (só http/https), resolver o host e **bloquear IPs privados/loopback/link-local** (`127.0.0.1/10.0.0.0/172.16.0.0/192.168.0.0`), limitar redirects, allowlist de domínios quando possível.

M-2 · SSRF / roubo de API key via `base_url` de provedor

Evidência: admin define `base_url` livre (`admin.py:188-221`); esse `base_url` recebe POST com `Authorization: Bearer <api_key decifrada>` (`llm.py:54-71, embeddings.py:60`). **Impacto:** um admin (ou admin comprometido via C-1) aponta o `base_url` para host próprio e **captura a chave real** enviada no header, ou usa a API como proxy SSRF autenticado. Requer privilégio admin → Média. **Correção:** allowlist de hosts de provedores conhecidos; validar esquema/host; nunca enviar credenciais para `base_url` arbitrário.

M-3 · Proteção de rotas do frontend só checa presença do cookie

Evidência: `middleware.ts:20-35` verifica apenas **existência** de `duofy_token`, não valida assinatura/expiração/role. Um cookie forjado qualquer (`duofy_token=x`) passa o gate do frontend. **Impacto:** o gate do front é cosmético; a segurança real está no backend (bom). Combinado com A-4 (bypass de middleware), reforça que não se deve confiar no middleware. Risco real baixo porque o backend valida — por isso Média/defesa em profundidade. **Correção:** tratar o middleware como UX; garantir que **toda** rota de API sensível exija `get_current_user/require_admin` (já ocorre nas revisadas); validar o JWT no middleware se ele for usado como controle.

M-4 · Container da API roda como root

Evidência: `apps/api/Dockerfile:1-32` — nenhum USER; `uvicorn` (e worker e migrate, mesma imagem) rodam como **root**. Contraste: `apps/web/Dockerfile.prod:29,35` usa USER `nextjs`. **Impacto:** uma RCE no container (via deps, WeasyPrint/Chromium, parsing de PDF/DOCX de upload) executa como root, ampliando o blast radius e facilitando escape. **Correção:** criar usuário não-root no `apps/api/Dockerfile` e USER antes do CMD; considerar `read_only/cap_drop` no compose.

M-5 · Chromium com `--no-sandbox` processando URLs não confiáveis

Evidência: `research_service.py:127-130` — `chromium.launch(args=["--no-sandbox", ...])` navegando em URLs externas/fornecidas pelo usuário (research "deep"). **Impacto:** `--no-sandbox` + conteúdo web hostil + processo root (M-4) = superfície séria de comprometimento do container por exploit de renderer. **Correção:** rodar o Chromium com sandbox (usuário não-root permite) ou isolar o fetch em processo/container dedicado com menos privilégios.

M-6 · CORS com `allow_credentials=True`

Evidência: `main.py:64-70` — `allow_credentials=True` com `allow_origins=settings.cors_origins`; `.env:5` usa localhost fixo. **Impacto:** hoje as origens são localhost (ok). Risco operacional: se `BACKEND_CORS_ORIGINS` virar * ou origem ampla com credenciais, habilita roubo cross-origin. O modelo atual usa Bearer, não cookie cross-site (atenua) — mas se C-1 migrar para cookie, isso escala para crítico. **Correção:** manter origens explícitas; validar em código que nunca haja * com credenciais.

BAIXA

B-1 · `Content-Disposition` com filename não sanitizado

`documents.py:71-76,234-238` — `filename` derivado do nome de upload do usuário, sem sanitizar `"/CRLF`. Impacto baixo (Starlette codifica boa parte). **Correção:** normalizar `filename` e usar `filename*=UTF-8''`.

B-2 · Possível log de dados sensíveis

`error_handlers.py:21-26` (`logger.exception`) e `llm.py:76-78` (inclui `exc.response.text[:600]` em mensagens que podem chegar a `AgentRun.error/Document.error`). A resposta ao cliente é genérica (bom), mas o corpo de erro de provedores pode gravar tokens/PII nos logs. **Correção:** redigir respostas de provedor antes de logar/persistir.

B-3 · PBKDF2 em vez de bcrypt/argon2 (informativo)

`security.py:15-34` — PBKDF2-HMAC-SHA256, 390k iterações, salt por senha, `compare_digest`. Implementação correta; não vulnerável. **Correção opcional:** migrar para `argon2id`, ou elevar iterações para ~600k (recomendação OWASP atual).

B-4 · Sem HSTS / CSP no Caddy

`infra/caddy/Caddyfile:19-24` — só `X-Content-Type-Options` e `Referrer-Policy`. Faltam `Strict-Transport-Security`, `Content-Security-Policy`, `frame-ancestors`. Sem HSTS há janela de downgrade/SSL-strip (agravado por permitir `:80`); sem CSP, o impacto de um XSS (C-1) é maior. **Correção:** adicionar HSTS (com domínio HTTPS), CSP restritiva e `frame-ancestors 'none'`.

[OK] Pontos positivos de segurança (confirmados)

- **Sem SQL injection:** o único raw SQL (`rag.py:55-92`) usa `text()` com parâmetros vinculados; `vector_to_sql` gera string numérica passada como parâmetro, não interpolada. O restante é ORM.
- **Sem path traversal em upload:** grava com `uuid4().hex + suffix` em `STORAGE_DIR`; o nome do usuário não influencia o caminho. Download usa `stored_path` do banco.
- **Segredos nunca expostos em API:** `password_hash` e `api_key_encrypted` jamais aparecem em schemas; provedores retornam `has_api_key/masked_api_key`.
- **Hardening de produção fail-fast:** `settings.py:44-65` recusa subir com JWT/admin default fora de dev.
- **Isolamento de rede em produção:** só o Caddy expõe portas; em dev, Postgres/Redis fazem bind só em `127.0.0.1`.
- **.env real sem segredos** e coberto pelo `.gitignore`; YAML sempre via `yaml.safe_load`.
- **Chat escapado por usuário** (contraste que evidencia o gap A-2).

Resumo de correção priorizada

Prioridade	Itens	Efeito
P0 (bloqueadores)	C-1, C-2	Fecha os dois vetores exploráveis remotamente sem privilégio.
P1	A-4, C-3	Elimina bypass de middleware; permite rotação de segredo sem incidente.
P2	A-1, A-2, M-1, M-4, M-5	Revogação de token; autorização de objeto; contenção de SSRF/RCE.
P3	A-3, M-2, M-3, M-6, B-1..B-4	Hardening de defesa em profundidade e higiene.

Detalhes de esforço e sequência em **07 · Plano de Ação Priorizado**.

04 · Modelo de Dados

Stack: SQLAlchemy 2 (async/asyncpg) + PostgreSQL 16 + pgvector + Alembic. **Veredito:** esquema **REAL e coeso** (24 tabelas, 24 modelos, nenhum órfão). Cadeia de 15 migrations linear e íntegra. Dívidas concentradas em **integridade referencial** (sem cascades, `brand_slug` sem FK) e **um índice faltante**.

1. Inventário por domínio

Base: `db.py:11` (`Base(DeclarativeBase)`). Mixin de timestamps (`models.py:22-33`): `created_at/updated_at` com `timezone=True`, `server_default=now()`, `updated_at onupdate=now()` (só client-side — ver §5).

Domínio	Tabelas	Confiança
Auth	<code>users</code> , <code>brands</code> , <code>settings</code>	REAL
Agentes/Credenciais	<code>agents</code> , <code>provider_credentials</code> , <code>agent_runs</code>	REAL
Memória/RAG	<code>sources</code> , <code>documents</code> , <code>document_chunks</code> , <code>memory_entries</code>	PARCIAL
Conteúdo/Outputs	<code>outputs</code> , <code>output_versions</code> , <code>output_decisions</code> , <code>output_comments</code>	REAL
Pesquisa	<code>research_sources</code>	REAL
Calendário	<code>calendar_events</code>	REAL (falta índice <code>category</code>)
Métricas/Relatórios	<code>model_calls</code> , <code>reports</code>	REAL
Tarefas/Chat	<code>chat_sessions</code> , <code>agent_tasks</code> , <code>chat_messages</code> , <code>agent_logs</code>	REAL
Auditoria	<code>audit_events</code>	REAL
Quality	<code>quality_reviews</code>	REAL

Chaves de negócio

- `Brand.slug` é a **chave de negócio de facto** — ~14 tabelas referenciam `brand_slug String(120)` sem FK (desacoplamento por slug; ver §5).
- `provider_credentials.api_key_encrypted` é nullable e `is_enabled` default `False` — seed não commita segredo.
- Colunas vetoriais: `document_chunks.embedding` e `memory_entries.embedding`, ambas `Vector(1536)` nullable.

Segurança de exposição (correta): `password_hash` e `api_key_encrypted` **nunca** aparecem em schemas Pydantic. `ProviderCredentialRead` expõe `has_api_key: bool` + `masked_api_key` (`schemas.py:56-64`).

2. Migrations — consistência e cadeia

- Cadeia `down_revision` íntegra e linear:** `0001 (None)` → ... → `0015`. Sem ramificações. Detalhe estético: o `revision` interno nem sempre bate com o nome do arquivo (ex.: `0003_provider_credentials_agent_runs.py` tem `revision="0003_provider_creds_runs"`), mas as referências fecham corretamente.
- `env.py:17, 50-51` usa `Base.metadata` com `compare_type=True` e `compare_server_default=True` — autogenerate funcional.

Divergências ORM ↔ migrations (achados)

#	Divergência	Evidência	Efeito
A	<code>calendar_events.category</code> tem <code>index=True</code> no ORM mas o índice não é criado na migration	<code>models.py:330</code> vs <code>0008:54-64</code>	Filtro por categoria faz seq scan; autogenerate vai querer criar o índice
B	<code>embedding</code> criado como <code>Text</code> + <code>ALTER ... TYPE vector(1536)</code> via SQL cru	<code>0004:67-74</code> vs <code>models.py:386 (Vector(1536))</code>	Resultado final correto, mas autogenerate pode gerar diffs espúrios
D	<code>updated_at</code> onupdate só client-side (ORM)	<code>models.py:31</code>	UPDATE via SQL cru não atualiza <code>updated_at</code> — armadilha real, há bastante <code>text()</code> no projeto

Defaults (`server_default` nas migrations vs `default` no ORM) são coerentes nos casos verificados (`role`, `passed`, colunas JSON `default=list`).

3. pgvector / RAG

- **Extensão** habilitada de forma redundante/defensiva em 3 lugares (init SQL, `0001`, `0004`), idempotente.
 - **Dimensão 1536** consistente em **todo** o sistema: `Vector(1536)` no ORM, `ALTER ... vector(1536)` nas migrations, `EMBEDDING_DIMENSIONS=1536` (`embeddings.py:13`), casado com `text-embedding-3-small` (seed).
- Alinhamento total.**
- **Índices ANN:** migration `0015` cria **dois índices HNSW** com `vector_cosine_ops` (`ix_document_chunks_embedding_hnsw`, `ix_memory_entries_embedding_hnsw`). Escolha correta (melhor recall/latência que ivfflat, sem tuning de lists).
 - **Operador casado:** `rag.py:68,84` usa `1 - (embedding <=> CAST(:embedding AS vector))` — `<=>` é cosseno, exatamente o que `vector_cosine_ops` indexa.

(!) Ressalva de performance (PARCIAL)

A query de busca (`rag.py:55-91`) faz `SELECT ... FROM (subquery1 UNION ALL subquery2) ranked ORDER BY score DESC LIMIT :limit`. O `ORDER BY score` está na **query externa** sobre um `UNION ALL`, e o `score` é a expressão `1 - (... <=> ...)`, não a distância pura ordenada por tabela. O índice HNSW só acelera quando o `ORDER BY <distância>` está diretamente na tabela indexada. **Portanto o planner provavelmente NÃO usa os índices HNSW nesta query** — computa a distância para todas as linhas filtradas e ordena. Os índices de `0015` existem mas podem não estar sendo aproveitados. **Ação:** rodar `EXPLAIN (ANALYZE, BUFFERS)` na query real; se confirmar seq scan, reescrever para aplicar `ORDER BY embedding <=> :q LIMIT k` por tabela (subconsulta indexável) e só então combinar/re-ranquear.

Fallback de embedding

`embeddings.py:29-41` gera vetor determinístico via hashing SHA-256 bucketizado, normalizado (coerente com cosseno). `_fit_dimensions (:21-26)` faz pad/trunca para 1536 **silenciosamente** — um provider de 3072-D seria truncado sem aviso, degradando qualidade. Ver [02 §1.12](#).

4. Índices e performance

Bem indexado para dashboards/filtros: `outputs` (5 índices), `calendar_events` (7), `model_calls` (6), `audit_events` (7). Boa disciplina.

FKs sem índice (dívida latente — Postgres não indexa FK automaticamente): `documents.source_id`, `memory_entries.source_id`, `outputs.agent_run_id`, `output_decisions.user_id/memory_entry_id`, `output_comments.user_id`, `quality_reviews.agent_run_id`, `chat_messages.agent_task_id`, `calendar_events.output_id/agent_run_id`. Um DELETE no lado pai faz seq scan nas filhas para checar as FKs. Aceitável no volume atual (equipe única), mas é dívida.

N+1: não há `relationship()` em nenhum model (zero lazy-loading ORM). Os agregados (`ChatSessionDetail.messages`, `ContentOutputDetail.versions` etc.) são montados por queries explícitas nos services — elimina o N+1 automático, mas transfere ao dev a responsabilidade de não fazer loop de queries manual.

5. Integridade

- **UNIQUE:** `users.email`, `brands.slug`, `agents.slug`, `settings.key`, `provider_credentials.provider`, e composto `uq_output_versions_output_version(output_id, version_number)`. Bom. (Faltaria talvez `research_sources(output_id, url)` para evitar fontes duplicadas.)
 - **FKs "lógicas" como Integer puro (integridade não aplicada) — 4 casos:** 1. `Output.current_version_id` (`models.py:177`) — dívida real (ciclo com `output_versions.output_id`). 2. `AgentTask.output_id`, 3. `ModelCall.task_id`, 4. `AuditEvent.entity_id` — **polimórficos intencionais** (o tipo vem de outra coluna), aceitáveis.
 - **`brand_slug` nunca é FK** — deletar/renomear uma marca deixa registros órfãos silenciosos em ~14 tabelas. Dívida de médio porte, provavelmente consciente.
 - **CASCADES: nenhum `ondelete/onupdate` em nenhuma FK.** Default `NO ACTION` → deletar um `output` com `versions/comments/reviews/decisions/research_sources` filhos **falha por violação de FK** a menos que o app delete os filhos manualmente em ordem. **Dívida real:** limpeza 100% na mão da aplicação.
 - **Timestamps** presentes em todas as tabelas; `updated_at` não atualiza em UPDATE via SQL cru (§2D).
-

6. Schemas Pydantic vs Models

- **Sem vazamento de segredo** (confirmado §1).
 - **Enums frouxos no banco, restritos no schema:** `role/status` são `String` livre no banco mas `Literal[...]` nos schemas (ex.: `UserRead.role: Literal["admin", "manager"]`, `schemas.py:22`). Um valor fora do enum no banco **quebra a serialização**. Risco baixo (seed usa valores válidos), mas real — o enum não é forçado no banco.
 - **Campos "virtuais" em DTOs de resposta:** `ContentOutputRead` expõe `current_version_number`, `current_content`, `document_type`, `document_sections`, `quality_notes` — nenhum existe na tabela; são montados no service. Legítimo, mas cria acoplamento (quebrar a montagem quebra o contrato).
 - **Validação saudável:** `Field(min_length/max_length/ge/le)` bem usado nos schemas de escrita; `EmailStr` em login/user.
-

7. Pool de conexão async (`db.py`)

`create_async_engine` (`db.py:16-24`): `pool_pre_ping=True`, `pool_size=10`, `max_overflow=20` (até 30 conexões/processo), `pool_timeout=30`, `pool_recycle=1800`, `expire_on_commit=False` (correto e necessário em async). `get_db()` usa `async with` (sessão sempre fechada).

Riscos em produção: 1. 30 conexões × N workers × (API + Celery) pode estourar o `max_connections` default do Postgres (100). Dimensionar com pgbouncer ou reduzir `pool_size` em prod. 2. `asyncpg` + pgbouncer em transaction mode exigiria `statement_cache_size=0` (não configurado) — dívida latente se um dia colocarem pgbouncer.

8. Seed (`seed.py`)

Idempotente (upsert por chave natural). Popula admin (role=admin, senha via hash_password), brands/agents (de YAML), settings, e 5 provider credentials todas com is_enabled=False e sem api_key (openrouter, anthropic, openai, openai_embeddings, apify). Nenhum segredo hardcoded. Hardening de produção bloqueia senha admin default fora de dev.

Importante (operacional): o seed é um script de CLI (python -m app.seed) e não roda automaticamente no boot do Docker. Ver 06 · Infra §4.

Sumário de achados acionáveis (modelo de dados)

#	Sev.	Achado	Local
1	Média	Índice calendar_events.category do ORM não existe na migration	models.py:330 vs 0008
2	Média	Query RAG ordena sobre UNION ALL → HNSW provavelmente não usado (confirmar c/ EXPLAIN)	rag.py:55-91
3	Média	Sem ON DELETE CASCADE em nenhuma FK → deletes bloqueiam/exigem limpeza manual	todas as migrations
4	Média	brand_slug sem FK em ~14 tabelas → órfãos silenciosos	esquema global
5	Baixa-Média	8+ FKs sem índice	§4
6	Baixa	updated_at onupdate só client-side → UPDATE cru não toca updated_at	models.py:31
7	Baixa	embedding como Text+ALTER → diffs espúrios no autogenerate	0004:67-74
8	Baixa	_fit_dimensions trunca embeddings silenciosamente	embeddings.py:21-26
9	Baixa	Pool 30 conn/processo pode estourar Postgres sem pgbouncer	db.py:16-23
10	Info	Enum String no banco vs Literal no schema → risco de erro de serialização	models.py:43 vs schemas.py:22

Pontos fortes: dimensão 1536 100% consistente; HNSW+vector_cosine_ops casado com <=>; cadeia de migrations linear e íntegra; seed idempotente e seguro; expire_on_commit=False correto; sem N+1 ORM; zero modelos mortos; nenhum segredo exposto.

Continue por 05 · Qualidade, Código Morto e Testes.

05 · Qualidade, Código Morto e Testes

Foco: alinhamento front↔back, código morto/desnecessário, cobertura de testes, consistência e estado da documentação.

Notas de síntese: - Alinhamento do código: 8,0 / 10 — contrato front↔back sólido, nenhuma chamada quebrada. - **Confiança na base de testes: 2,0 / 10 (~20%)** — boa cobertura unitária de lógica isolada, **nenhum teste E2E/integração/DB/frontend, nenhuma CI.**

1. Alinhamento Frontend ↔ Backend

Todas as chamadas `apiFetch/downloadFile` do frontend foram mapeadas contra as ~78 rotas do backend.

[OK] **Nenhum endpoint chamado pelo front que não exista no back.** Todos os paths (`/api/auth/*`, `/api/outputs/*`, `/api/content/*`, `/api/research/*`, `/api/calendar/*`, `/api/memory/*`, `/api/documents/*`, `/api/metrics/*`, `/api/reports/*`, `/api/operations/*`, `/api/admin/*`, `/api/agents/*`, `/api/chat/*`, `/api/tasks/*`, `/api/brands`) têm rota correspondente. A tipagem em `lib/api.ts` casa com os `response_model`.

Rotas do backend que NINGUÉM consome (código órfão de servidor)

Rota	Local	Situação	Severidade
POST <code>/api/press/generate</code> (router inteiro)	<code>routers/press.py:57</code>	Assessoria de imprensa só existe no back; nenhuma tela chama	Dívida
PATCH <code>/api/agents/runs/{id}/status</code>	<code>routers/agents.py:95</code>	Nunca chamado (aprovação real via <code>/api/outputs/*</code>)	Dívida
GET <code>/api/outputs/{id}/pdf</code>	<code>routers/outputs.py:642</code>	Órfã: front usa <code>/export?format=pdf</code>	Limpeza
GET <code>/api/reports/{id}/pdf</code>	<code>routers/reports.py:104</code>	Órfã: front usa <code>.../export</code>	Limpeza
POST <code>/api/outputs/reformat-legacy</code>	<code>routers/outputs.py:464</code>	Manutenção; já roda no boot	Limpeza
POST <code>/api/outputs/repair-formatting</code>	<code>routers/outputs.py:495</code>	Manutenção; sem UI	Limpeza

Inconsistência de UX de erro: o handler global devolve JSON `{"error","detail"}` (`error_handlers.py:21-26`), mas `apiFetch` lê o corpo via `response.text()` e joga a string crua no `Error` (`api.ts:428-431`) — em 500 o usuário vê o JSON literal, não a mensagem.

2. Código morto / desnecessário

2.1 Frontend — 5 componentes órfãos (~2.368 linhas) — o achado mais grave

O último commit (`9dfa586`) reescreveu `/operations` com kanban, drag-drop, chat e modal **inline próprios** (tudo self-contained). Com isso, os componentes do commit anterior (`3bcac4f`) ficaram **sem nenhum consumidor**:

Componente	Linhas	Importado por
<code>components/kanban-board.tsx</code>	437	ninguém
<code>components/card-popup.tsx</code>	546	ninguém

<code>components/chat-panel.tsx</code>	549	ninguém
<code>components/inspector-bar.tsx</code>	195	ninguém
<code>components/document-workspace.tsx</code>	641	só o tipo <code>ExportFormat</code> (usado por <code>lib/download.ts:1</code>) — morto por transitividade quanto ao componente
<code>components/logout-button.tsx</code>	24	ninguém (logout real está inline no <code>app-shell</code>)

`operations/page.tsx` importa apenas `ui`, `markdown`, `icons`, `auth`, `brand-context`, `download`. Confirmação extra de que são resquícios: `chat-panel.tsx:103-108` e `inspector-bar.tsx:140-145` referenciam rotas que **não existem mais** (`/content/:id`, `/research/:id`, `/reports/:id`).

Reconciliação de divergência entre auditorias: uma frente inicialmente marcou esses componentes como "possivelmente órfãos, verificar" e afirmou que `/operations` usava `chat-panel`. A verificação direta por `grep` refuta isso — **são código morto**. Prevalece a leitura verificada.

2.2 Frontend — `lib/mock.ts` (489 linhas): 100% morto

Zero importadores (`grep "@/lib/mock"` em todo `apps/web` = 0). A única tela mock (`/redes`) define seus próprios dados hardcoded localmente, nem importa de `mock.ts`. **Deletar sem risco.**

2.3 Frontend — 7 páginas viraram stubs de `redirect`

`dashboard`, `insights`, `workspace`, `costs`, `content`, `research`, `chat` (cada uma 4 linhas → `/operations` ou `/relatorios`). Inofensivas, mas o `middleware.ts:3-18` e `config.matcher:39-55` ainda protegem essas 9 rotas mortas.

2.4 Frontend — páginas admin órfãs

`/admin/agents` (290 linhas) e `/admin/config` (417 linhas) são **totalmente funcionais** mas **sem link na navegação** (o sidebar vai só para `/admin`). Duplicam funcionalmente as abas de `/admin`. Decidir: remover ou re-linkar.

2.5 Backend — `pdf_service.py` órfão

22 linhas, wrapper fino de `export_service.build_duofy_pdf`, **zero importadores**. Deletar.

2.6 Backend — funções redefinidas/sombreadas em `routers/outputs.py`

`_version_read` (L60 e L256), `_output_read` (L100 e L267), `_output_export_document` (L190 e L301) são **definidas duas vezes** — a segunda (com `repair_text`) sobrescreve a primeira em runtime; a primeira é **código morto**. Além disso, `_quality_review_read` usa `repair_text` em `outputs.py` mas **não** em `content.py` — mesma entidade renderizada diferente conforme o endpoint.

2.7 Backend — duplicação de helpers

Função	Cópias	Locais	Recomendação
<code>_provider_for_model</code>	5	<code>calendar_service</code> , <code>content_generation</code> , <code>orchestrator</code> , <code>orchestrator_graph</code> , <code>quality_guardian</code>	Consolidar em <code>llm.py</code> . Divergência real: só a de <code>orchestrator.py</code> trata prefixo <code>~anthropic/</code> ; as outras 4 não → risco de drift.
<code>_plain_text/_plain</code>	4-6	<code>calendar_service</code> , <code>content_generation</code> , <code>orchestrator</code> , <code>quality_guardian</code>	Mover para <code>text_repair.py</code> .

<code>_system_prompt</code>	3–4 variações	<code>orchestrator</code> , <code>content_generation</code> , <code>calendar_service</code> , <code>research_service</code>	Assinaturas divergem; avaliar unificação.
descoberta de <code>config/</code>	3	<code>agent_config</code> , <code>metrics</code> , <code>output_workflow</code>	Um único loader.

~250 linhas duplicadas no back; principal risco é corrigir num lugar e esquecer nos outros.

2.8 O que NÃO é duplicação (esclarecimentos)

- `metrics.py` (instrumentação) vs `metrics_service.py` (agregação/consulta) — camadas complementares corretas.
- `content_generation.py` (serviço) vs `routers/content.py` (router) — separação serviço/router correta.
- Os 4 orquestradores estão todos ativos e encadeados (nenhum órfão): `orchestrator.py`←agents/calendar; `orchestrator_graph.py`←worker; `orchestrator_llm.py`/`orchestrator_tools.py`←grafo.
- `seed.py` não é morto — é script de CLI documentado (`python -m app.seed`); correto não ser importado.

2.9 Arquivos versionados — situação limpa

`.env` (raiz) só tem `BACKEND_CORS_ORIGINS` (sem chaves) e está gitignorado. `tmp/`, `.superpowers/`, caches e `tsbuildinfo` não estão tracked. Sem segredos vazados no git.

2.10 Bug de estilo — classe Tailwind `surface` inexistente

`bg-surface/hover`: `bg-surface` usada 17× em 5 arquivos (`admin/page.tsx` ×11, `redes`, `operations`, `calendar`, `markdown`), mas `surface` não está definida em `tailwind.config.ts` (que define `panel/linen/ink`) nem em `globals.css`. As classes são silenciosamente ignoradas — o hover/background pretendido não aparece. Corrigir: definir `surface` ou trocar por `panel`.

Total removível/limpável

~2.850 linhas de código morto no frontend (`mock.ts` 489 + cluster órfão ~2.360) + `pdf_service.py` + funções sombreadas no back + ~250 linhas de helpers duplicados. Remoção de baixo risco, alto ganho de legibilidade.

3. Testes

78 funções de teste em 22 arquivos (`apps/api/tests/*`), todos backend, quase todos unitários com fakes/mocks.

Bem coberto

`test_settings` (fail-fast de segredos), `test_quality_guardian` (rubrica local + híbrido + fallback), `test_agent_limits/test_agent_budgets_applied` (budgets/depth), `test_llm_retry/test_llm_max_tokens` (retry 429/5xx, max_tokens), `test_calendar_ics/test_calendar_parse` (ICS + parse robusto), `test_text_repair` (mojibake PT-BR), `test_export_pdf` (PDF/markdown/XSS), `test_brand_profile` (voz de marca), `test_orchestrator_*` (graph/tools/prompt/llm/integration).

Não coberto (áreas críticas)

- [FALTA] Zero testes de HTTP/integração de negócio autenticado. Só 4 arquivos usam `TestClient` e nenhum exercita fluxo real.
- [FALTA] Zero DB real — tudo é `_FakeSession`; não há `conftest.py`.
- [FALTA] Fluxo de aprovação de output (`outputs.py`:695-804, `output_workflow.py`) — núcleo do produto — 0 testes.

- [FALTA] **Auth/JWT completo** (login → token → rota protegida) — só há teste de "faltou token".
- [FALTA] **Worker Celery / task_service / chat sessions** — 0.
- [FALTA] **RAG/embeddings/rag.py/document_processing.py** (upload + chunking + busca) — 0.
- [FALTA] **crypto.py** (Fernet das credenciais) — 0.
- [FALTA] **calendar_scheduler.py** — 0.
- [FALTA] **Frontend: 0 testes** (nenhuma dep de vitest/jest/playwright; nenhum *.test.*).
- [FALTA] **CI: inexistente** — sem .github/workflows, sem pipeline. Nada roda testes em push/PR.

Confiança estimada: ~20%

Valida bem lógica pura isolada; **não valida nenhum caminho ponta-a-ponta**, persistência, worker nem UI. Qualquer regressão em routers, `output_workflow` ou worker passa despercebida.

4. Consistência e convenções

- **Padrão serviço/router** bem aplicado no back (routers finos delegando a serviços). Positivo.
- **Tratamento de erro variável:** handler global consistente, mas `raise HTTPException/except` espalhados com granularidade desigual (`outputs.py` tem 14; `auth.py/chat.py` têm 1). No front, tratamento de erro divergente (`window.alert/window.prompt` em approvals vs modais em operations vs estado silencioso em memory).
- **Resiliência de fetch inconsistente:** `Promise.allSettled` vs `Promise.all` (padronizar no primeiro).
- **API_URL duplicado** em 4 arquivos (`api.ts`, `download.ts`, `memory/page.tsx`, `chat-panel.tsx`) — centralizar.
- **Tipos redefinidos:** `admin/page.tsx:13-60` redefine localmente `Agent/Provider/AgentRun/etc.` que já existem em `lib/api.ts` — importar de lá.
- **Casts frouxos:** `as unknown as ResearchReport` (`operations/page.tsx:493,497`) — usar type guard.
- **Nomenclatura PT/EN misturada** nas rotas (`/operations`, `/relatorios`, `/redes`, `/approvals`) — cosmético.

5. TODO/FIXME/HACK/XXX

Nenhum marcador real no código-fonte (matches de grep foram falsos positivos por substring, ex.: "todos", "metodologia"). Código limpo nesse aspecto.

6. Documentação vs Código

Documento	Estado	Ação
<code>docs/ESTADO_PROJETO_2026-06-26.md</code>	Preciso e atual — melhor referência (LangGraph, HNSW, Tier 0, mono-tenant)	Manter como fonte de verdade
<code>docs/ESTADO_ATUAL_DO_SISTEMA.md</code> (65 KB)	Desatualizado — diz migrations até 0014 (repo está em 0015)	Arquivar/remover
<code>docs/MAPA_TECNICO_E_OPERACIONAL.md</code> (51 KB)	Sobreposto ao anterior, parcialmente estale	Consolidar
<code>docs/SPRINT_FINAL_STATUS.md</code>	Números estale ("14 passed", "17 rotas"; hoje 78 testes)	Atualizar/arquivar
<code>README.md</code> (raiz)	Desatualizado — lista <code>/dashboard</code> , <code>/costs</code> , <code>/insights</code> como telas (hoje redirects); descreve "Fase 1/2"	Reescrever seção de rotas

IMPLEMENTATION_STATUS.md	Coerente em alto nível, mas não reflete o /operations unificado	Atualização leve
DEPLOY.md	Recente e alinhado com a stack prod — exceto a afirmação de seed automático (ver 06)	Corrigir passo de seed

Nenhum doc menciona o `press_agent` sem UI nem os 5 componentes órfãos — a documentação não acompanhou os 2 últimos commits de redesign.

Nota: a memória do projeto já registra "Doc ESTADO_ATUAL desatualizado" — verificar sempre no código antes de agir sobre o que os docs afirmam.

7. Top 5 ações de limpeza (ordem de impacto)

1. Remover os 5 componentes órfãos + `lib/mock.ts` (~2.850 linhas) — ou re-cablá-los se o redesign os previa.
2. Adicionar **CI mínima** (ruff + pytest + `next lint/build`) — hoje nada roda automaticamente.
3. Criar `conftest.py` com DB de teste + ~5 testes de integração (login JWT e fluxo de aprovação).
4. Consolidar `_provider_for_model` (5x) e `_plain_text` (4x) em módulos únicos.
5. Deletar `pdf_service.py`; remover funções sombreadas de `outputs.py`; corrigir a classe `surface`; arquivar docs estale; corrigir seção de rotas do `README.md`.

Continue por **06 · Infraestrutura e Deploy**.

06 · Infraestrutura e Deploy

Escopo: stack Docker (dev + prod), build, migrations, TLS/Caddy, config por ambiente, observabilidade, resiliência e prontidão para VPS. **Veredito global: PARCIAL.** A topologia, o TLS e o hardening de segredos são REAIS e bem-feitos; mas há **um gap de bootstrap** (seed do admin não roda no boot) e lacunas operacionais (sem backup automatizado, worker single-threaded, sem healthcheck no worker/Caddy, sem limites de recurso) que impedem classificar como pronto.

1. Topologia

DEV — docker-compose.yml

Serviço	Imagem/Build	Portas	Healthcheck	Restart
postgres	pgvector/pgvector:pg16	127.0.0.1:5433→5432	pg_isready	unless-stopped
redis	redis:7-alpine	127.0.0.1:6379	redis-cli ping	unless-stopped
migrate	api Dockerfile	—	—	no (one-shot)
api	api Dockerfile	8000→8000	urllib /health	unless-stopped
web	apps/web/Dockerfile (next dev)	3000→3000	wget /login (IPv4)	unless-stopped
worker	api Dockerfile	—	nenhum	unless-stopped

Boot: postgres/redis healthy → migrate completed → api+worker → web (espera api healthy).
Postgres/Redis só em loopback.

PROD — docker-compose.prod.yml

Serviço	Portas	Volumes	Healthcheck
postgres	nenhuma	postgres_data	pg_isready
redis	nenhuma	redis_data (AOF)	redis-cli ping
migrate	—	—	— (restart: no)
api	nenhuma	storage_data	urllib /health
worker	—	storage_data	nenhum
web	nenhuma	— (Dockerfile.prod)	wget /login
caddy	80:80, 443:443	Caddyfile ro, caddy_data, caddy_config	nenhum

Exposição prod (ponto forte): só o Caddy publica portas. Postgres/Redis/API/worker/web ficam exclusivamente na rede interna do Docker.

2. Dev vs Prod

Aspecto	Dev	Prod
Web	npm run dev (hot-reload, bind do código)	Next standalone non-root (node server.js)

Reverse proxy	ausente	Caddy + TLS automático
Banco/Redis	loopback do host	rede interna, zero portas
APP_ENV	development	production
Segredos	defaults tolerados	obrigatórios via <code>\${VAR:?}</code> + validados na API
NEXT_PUBLIC_API_URL	http://localhost:8000	vazio → <code>/api</code> relativo via Caddy (same-origin, sem CORS)
Storage	bind <code>./storage</code>	volume <code>storage_data</code>

A stack de prod é **self-contained e coerente** — referencia só arquivos versionados + `.env`. A coerência quebra **só no bootstrap** (§4).

3. Build

API (apps/api/Dockerfile): - Single-stage (`python:3.11-slim`) — imagem grande (carrega toolchain). - [OK] **Libs WeasyPrint/PDF presentes** (:9-19): `libpango`, `libpangocairo`, `libcairo2`, `libgdk-pixbuf`, `libffi8`, `shared-mime-info` + fontes. **O PDF funciona.** - (!) `playwright install --with-deps chromium` (:23) baixa Chromium + libs (centenas de MB). Somado a Pango/Cairo, a imagem api/worker/migrate é pesada → requisito de **≥4 GB RAM** (DEPLOY.md). - Cache correto: `requirements.txt` antes do código. - (!) **Sem lock:** `requirements.txt` usa faixas (`>=x`, `<y`, alguns sem teto) → builds não reproduzíveis entre datas.

Web prod (Dockerfile.prod): - [OK] Multi-stage (`deps`→`builder`→`runner`), imagem enxuta, **non-root** (`USER nextjs`), `output: "standalone"` casado com `next.config.mjs:12`. - (!) Sem lockfile próprio (`npm install`, não `npm ci`) → build não 100% reproduzível.

4. (!) Bootstrap do admin — GAP OPERACIONAL (Alta)

Migrations em si: corretas e idempotentes. O serviço `migrate` roda `alembic upgrade head` e sai; api/worker só sobem após `service_completed_successfully`. 15 revisions lineares; pgvector habilitado no init SQL e na `0001`.

O problema — o seed do admin não roda automaticamente: - `DEPLOY.md:52-54` afirma que *"o primeiro boot semeia o admin, marcas e agentes"*. - Na realidade, `seed.py:152-153` só executa via `if __name__ == "__main__"`. Grep confirma: `seed_*` **não é chamado no runtime** — nem no `migrate` (comando é só `alembic upgrade head`), nem no lifespan da API (`main.py:41-54` só chama `reformat_legacy_outputs`), nem em entrypoint/CMD. - **Consequência:** num deploy limpo seguindo o `DEPLOY.md`, **não existe usuário admin** — o login inicial falha, mesmo com a stack "saúdável".

Reconciliação: o `seed.py` **não é código morto** — é um script de CLI válido e o `README.md` documenta `docker compose exec api python -m app.seed`. O gap é a **contradição:** o `DEPLOY.md` promete automático, mas é manual. Ambas as leituras (uma auditoria disse "seed nunca roda no boot", outra disse "seed é CLI válido") estão corretas e compõem o mesmo achado.

Correção recomendada (idempotente, o seed já suporta):

```
# serviço migrate
command: ["sh", "-c", "alembic upgrade head && python -m app.seed"]
```

Alternativas: chamar `seed.main()` no lifespan da API, ou no mínimo corrigir o `DEPLOY.md` com o passo manual explícito.

5. TLS / Caddy (prod)

infra/caddy/Caddyfile: - **Endereço dinâmico:** `{\$DUOFY_SITE_ADDRESS::80}` — domínio → HTTPS automático (Let's Encrypt); `:80` → HTTP puro (teste por IP). Idiomático. - **Roteamento:** `/api/* /health` → `api:8000`; resto → `web:3000` (same-origin, sem CORS). - `encode gzip zstd`; remove header `Server`; adiciona `X-Content-Type-Options` e `Referrer-Policy`. - Certificados persistidos em `caddy_data` (sobrevivem a restart). - (!) **Faltam HSTS e CSP** (ver 03 B-4) e healthcheck no container Caddy.

6. Config por ambiente

Hardening de segredos: REAL e bem-feito (defesa em profundidade): - `settings.py:44-65` — com `APP_ENV` fora de dev, a API **recusa subir** se `JWT_SECRET_KEY` for default/`change-me`, tiver <32 chars, ou `ADMIN_PASSWORD` for default. - `docker-compose.prod.yml` reforça com `${VAR:?}` para `POSTGRES_PASSWORD`, `JWT_SECRET_KEY`, `ADMIN_EMAIL/PASSWORD`, `DUOFY_SITE_ADDRESS`. - `.env.production.example` alerta corretamente sobre o acoplamento `JWT_SECRET_KEY`↔Fernet na migração.

Defaults que só afetam dev: `JWT_SECRET_KEY=change-me...`, `ADMIN_PASSWORD=admin123456` — bloqueados em prod pelo guard. `ACCESS_TOKEN_EXPIRE_MINUTES=720` (12h) em ambos — combinado com o JWT não-HttpOnly, é janela longa de exposição.

7. Observabilidade

- **Logs:** stdout via `logging.basicConfig` — capturados pelo Docker. **Sem log estruturado, sem rotação/limite** (`json-file` sem `max-size` cresce sem limite no disco da VPS).
 - **Healthchecks:** postgres, redis, api (`/health` valida pg+redis), web (`/login`). **Faltam no worker e no caddy.** Worker sem healthcheck = Celery travado não é detectado (fica "running" mas morto).
 - **Métricas:** existe `/api/metrics/summary` (negócio), mas **sem métricas de infra** (CPU/mem/latência/fila Celery).
 - **Restart policies:** `unless-stopped` (longa duração) e `no` (migrate) — corretas.
-

8. Resiliência e escala

- (!) **Worker --pool=solo:** processa **uma task por vez, sem paralelismo** (escolhido por causa de `asyncio.run` na task e Playwright/Chromium que não gostam de fork). Aceitável para equipe única, mas é gargalo — geração (LLM + scraping + PDF) serializa. Escalar exige réplicas do worker (não há `deploy.replicas`) ou trocar o pool com cuidado.
 - (!) **SPOFs:** postgres, redis, caddy são instâncias únicas — esperado em VPS single-node, sem HA.
 - (!) **Backup do Postgres NÃO automatizado:** o `DEPLOY.md §4` dá um script de cron de exemplo (`pg_dump` + tar + retenção 14d), mas ele **não é instalado por nada**. Seguir o guia à risca deixa o sistema sem backup até alguém configurar o cron. **Risco alto.**
 - [OK] **Persistência de volumes correta:** `postgres_data`, `redis_data` (AOF), `storage_data`, `caddy_data/config` são volumes nomeados. Procedimento de migração VPS→VPS do `DEPLOY.md` é detalhado e sólido (dump custom + restore + tar do storage + alerta Fernet↔JWT).
 - (!) **Sem limites de CPU/memória** em nenhum serviço. Com Chromium + WeasyPrint + LLM, um pico no worker pode consumir toda a RAM e derrubar o Postgres por OOM numa VPS de 4 GB.
-

9. Checklist de prontidão para produção

Bloqueadores

1. **Seed não roda no boot** → sem admin, sistema inacessível. Adicionar `python -m app.seed` ao `migrate` (idempotente) ou corrigir o `DEPLOY.md` com o passo manual.

Alta prioridade

2. **Backup automatizado do Postgres** — instalar de fato o cron/systemd timer (hoje só exemplo no doc).
3. **Limites de recurso** (`mem_limit/cpus` ou `deploy.resources`) ao menos para `worker` e `api` — evitar OOM.
4. **Healthcheck no worker** (`celery inspect ping`).

Média prioridade

5. **Rotação/limite de logs** (`logging: driver json-file, options max-size/max-file`).
6. **HSTS no Caddy** (produção com domínio).
7. **Escala do worker** — avaliar réplicas se o volume crescer.
8. **Reprodutibilidade de build** — pin/lock (`requirements.txt` com faixas; web via `npm install`).
9. **Non-root no container da API** (ver 03 M-4).

Dívidas conscientes (aceitas)

10. JWT cookie não-HttpOnly (mitigado por same-origin) — mas continua sendo o risco top de segurança.
11. `npm audit` Next.js 14 — upgrade em tarefa isolada.

[OK] Pontos fortes confirmados

- Exposição mínima em prod (só Caddy 80/443).
- Hardening de segredos fail-fast em dois níveis.
- TLS automático Let's Encrypt, same-origin sem CORS.
- Libs WeasyPrint/Pango/Cairo presentes → PDF funciona.
- Web prod standalone non-root, imagem enxuta.
- Boot determinístico (`service_completed_successfully/service_healthy`).
- Procedimento de migração VPS→VPS correto (incluindo alerta Fernet↔JWT).

Veredito: com o fix do seed + backup em cron + limites de memória + healthcheck do worker, a stack passa de **PARCIAL** para **REAL** para uma VPS single-node de equipe interna.

Continue por **07 · Plano de Ação Priorizado**.

07 · Plano de Ação Priorizado

Backlog consolidado a partir das 6 auditorias, ordenado por prioridade. Esforço em escala relativa: **P** (pequeno, < 0,5 dia) · **M** (médio, 0,5–2 dias) · **G** (grande, 2–5 dias).

Legenda de origem: [02] Funcionalidades · [03] Segurança · [04] Dados · [05] Qualidade · [06] Infra.

Fase 0 — Bloqueadores (antes de qualquer produção)

#	Ação	Esforço	Origem
0.1	Migrar JWT para cookie HttpOnly; Secure; SameSite=Strict emitido pelo backend; ler server-side; ajustar <code>getTokenFromCookie</code> nas telas	G	[03] C-1
0.2	Rate limiting + logout no /login (slowapi ou no Caddy) + remover credenciais default do form	M	[03] C-2
0.3	Automatizar o seed no boot (<code>alembic upgrade head && python -m app.seed</code> no serviço <code>migrate</code>) e corrigir o <code>DEPLOY.md</code>	P	[06] §4
0.4	Atualizar Next.js (14.2.x/15.x patchado) em branch isolada com regressão	M	[03] A-4

Resultado da Fase 0: fecha os dois vetores exploráveis remotamente (C-1, C-2), torna o deploy funcional (0.3) e elimina o bypass de middleware (0.4).

Fase 1 — Segurança estrutural

#	Ação	Esforço	Origem
1.1	Separar JWT_SECRET_KEY da chave de criptografia (<code>SECRETS_ENCRYPTION_KEY</code> via <code>Fernet.generate_key()</code>); suportar rotação com re-cifragem. Fazer antes de qualquer rotação.	M	[03] C-3
1.2	Escopar documentos por marca/usuário (<code>download/export/chunks/list</code>) — fechar IDOR/BOLA	M	[03] A-2
1.3	Revogação de token: reduzir expiração (30–60 min) + refresh token + <code>jti/denylist</code> em Redis	G	[03] A-1

1.4	Conter SSRF no research: validar esquema/host, bloquear IPs privados/loopback/link-local/metadata, limitar redirects	M	[03] M-1
1.5	Non-root no container da API + <code>cap_drop</code> ; rodar Chromium com sandbox	M	[03] M-4/M-5
1.6	Allowlist de host para <code>base_url</code> de provedor (evitar roubo de API key)	P	[03] M-2
1.7	HSTS + CSP + <code>frame-ancestors</code> no Caddy	P	[03] B-4

Fase 2 — Prontidão operacional

#	Ação	Esforço	Origem
2.1	Backup automatizado do Postgres (cron/systemd: <code>pg_dump</code> custom + tar do storage + retenção)	M	[06] §8
2.2	Limites de recurso (<code>mem_limit/cpus</code>) para <code>worker</code> e <code>api</code> — evitar OOM	P	[06] §8
2.3	Healthcheck no worker (<code>celery inspect ping</code>)	P	[06] §7
2.4	Rotação/limite de logs (<code>json-file</code> <code>max-size/max-file</code>)	P	[06] §7
2.5	Mover o scheduler de calendário para Celery beat (sair do processo web)	M	[02] §1.10
2.6	Pin/lock de dependências (<code>requirements</code> com teto; web <code>npm ci</code> com lockfile)	M	[06] §3

Fase 3 — Verificação (testes & CI)

#	Ação	Esforço	Origem
3.1	CI mínima (GitHub Actions): <code>ruff check</code> + <code>pytest</code> + <code>next lint</code> + <code>next build</code> em push/PR	M	[05] §3
3.2	<code>conftest.py</code> com DB de teste (Postgres efêmero/testcontainers)	M	[05] §3
3.3	Testes E2E do login JWT (login → token → rota protegida → 401)	M	[05] §3

3.4	Testes E2E do fluxo de aprovação (gerar → submit-review → Guardião → approve/reject)	G	[05] §3
3.5	Testes de RAG (upload → chunk → embed → busca) e worker/orquestrador	G	[05] §3
3.6	Smoke test de frontend (Playwright) das 7 telas reais	M	[05] §3

Fase 4 — Higiene de código (baixo risco, alto ganho)

#	Ação	Esforço	Origem
4.1	Deletar <code>lib/mock.ts</code> (489 linhas)	P	[05] §2.2
4.2	Deletar cluster órfão (<code>chat-panel</code> , <code>kanban-board</code> , <code>card-popup</code> , <code>inspector-bar</code> , <code>logout-button</code> ; reduzir <code>document-workspace</code> ao tipo <code>ExportFormat</code> ou movê-lo)	M	[05] §2.1
4.3	Decidir sobre <code>/admin/agents</code> e <code>/admin/config</code> (remover ou re-linkar)	P	[05] §2.4
4.4	Remover funções sombreadas de <code>routers/outputs.py</code> ; deletar <code>pdf_service.py</code>	P	[05] §2.5/2.6
4.5	Consolidar <code>_provider_for_model</code> (5×, com a divergência <code>~anthropic/</code>) e <code>_plain_text</code> (4×)	M	[05] §2.7
4.6	Corrigir a classe Tailwind <code>surface</code> inexistente (17 usos)	P	[05] §2.10
4.7	Enxugar <code>middleware.ts/matcher</code> das 9 rotas mortas; centralizar <code>API_URL</code> ; importar tipos de <code>lib/api.ts</code> no admin	P	[05] §4
4.8	Tratamento centralizado de 401 no <code>apiFetch</code> (logout automático) + padronizar <code>Promise.allSettled</code>	M	[02] §2

Fase 5 — Robustez de dados e RAG

#	Ação	Esforço	Origem
---	------	---------	--------

5.1	Confirmar com EXPLAIN ANALYZE se a query RAG usa o índice HNSW; se não, reescrever para ORDER BY embedding <=> :q LIMIT k por tabela	M	[04] §3
5.2	Adicionar ON DELETE CASCADE (ou lógica explícita de limpeza) nas FKs de outputs/filhos	M	[04] §5
5.3	Criar índice faltante ix_calendar_events_category + índices nas FKs mais quentes	P	[04] §2/§4
5.4	Avisar/validar quando o embedding do provider não for 1536-D (em vez de truncar em silêncio); embed_text com try/retry	P	[04] §3 / [02] §1.12
5.5	Decidir sobre brand_slug : FK real ou documentar o modelo desacoplado	M	[04] §5

Fase 6 — Funcionalidades e produto

#	Ação	Esforço	Origem
6.1	Upload de documentos na /memory (o backend já existe)	M	[02] §1.13
6.2	Decidir sobre assessoria de imprensa : expor UI para /api/press ou remover o router	M	[05] §1
6.3	Documentar/expor o override de modelo (default_model) sobrepõe a escolha do usuário — é bug ou feature?	P	[02] §1.17
6.4	Migrar o checkpointer do orquestrador de MemorySaver para AsyncPostgresSaver (estado durável)	G	[01] §3
6.5	Substituir mock de /redes por backend real (Meta Ads/Instagram) — ou marcar como "roadmap" na UI	G	[02] §2
6.6	Concluir integrações "Em breve" do admin (Permissões, Sheets, Tavily, SendGrid) conforme roadmap	G	[02] §2

Fase 7 — Documentação

#	Ação	Esforço	Origem
---	------	---------	--------

7.1	Corrigir seção de rotas do <code>README.md</code> (dashboard/costs/insights são redirects) e o passo de seed do <code>DEPLOY.md</code>	P	[05] §6
7.2	Arquivar docs estale (<code>ESTADO_ATUAL_DO_SISTEMA.md</code> , <code>SPRINT_FINAL_STATUS.md</code>); eleger <code>ESTADO_PROJETO_2026-06-26.md</code> como fonte de verdade	P	[05] §6
7.3	Atualizar <code>IMPLEMENTATION_STATUS.md</code> para refletir o <code>/operations</code> unificado	P	[05] §6

Roadmap sugerido (visão de sprints)

- **Sprint 1 — "Deploy seguro"**: Fase 0 completa + 1.1, 1.2 + 2.1, 2.2, 2.3. → sistema instalável e sem os vetores críticos.
- **Sprint 2 — "Confiança"**: Fase 3 (CI + testes E2E de login e aprovação) + Fase 4 (limpeza de código morto). → regressões passam a ser detectáveis; repositório enxuto.
- **Sprint 3 — "Robustez"**: restante da Fase 1, Fase 2 (2.4–2.6), Fase 5. → segurança em profundidade + RAG/dados sólidos.
- **Sprint 4+ — "Produto"**: Fase 6 conforme prioridade de negócio + Fase 7.

Definição de "pronto para produção" (Definition of Done)

1. Fase 0 e Fase 2 (2.1–2.3) concluídas.
2. C-3 (separação de segredos) e A-2 (IDOR) fechados.
3. CI verde com ao menos os testes E2E de login e aprovação.
4. Backup automatizado verificado (restore testado).
5. Documentação de deploy corrigida e validada num deploy limpo real.

Índice completo: [README](#) · Comece pelo [Sumário Executivo](#).